LAB GUIDELINES

Lab 6 — Installing a Server Operating System (Unattended / Scripted)

Maps to CompTIA Server+ objective **2.1 — Given a scenario, install server operating systems** (GUI, Core, Bare metal, Virtualized, Remote, Slip-streamed / unattended, scripted, additional drivers, media installation, imaging, cloning, P2V, template deployment, HCL).

Run all commands on https://killercoda.com/playgrounds/scenario/ubuntu

Required software (free, apt): `debootstrap`, `cloud-init`, `genisoimage`, `qemu-utils`

```
apt update && apt install -y debootstrap cloud-init genisoimage qemu-utils
```

Step 1 — Confirm minimum OS requirements

Before any install: minimum CPU, RAM, disk, and supported architecture.

```
lscpu | grep -E "Architecture|CPU\(s\)"
free -h
lsblk
```

Document what the OS vendor's minimum spec is (e.g. Ubuntu 22.04 server: 1 GHz CPU, 1 GB RAM, 2.5 GB disk) and verify yours is above it.

Step 2 — Hardware Compatibility List (HCL) check

```
echo "Product: $(dmidecode -s system-product-name)"
echo "CPU: $(lscpu | awk -F: '/Model name/{print $2}')"
echo "NIC: $(lspci | grep -i ether)"
echo "Storage controller: $(lspci | grep -i -E 'sata|raid|sas')"
```

Cross-check each against the vendor HCL (Red Hat, SUSE, Ubuntu, VMware). Going off-HCL is a top cause of post-install driver problems.

Step 3 — Unattended / scripted install — build an autoinstall config

Ubuntu Server 22.04 uses **cloud-init** for unattended installs (the modern replacement for preseed/kickstart). Build a minimal config:

```
mkdir -p /srv/autoinstall && cd /srv/autoinstall
cat > user-data <<'EOF'
#cloud-config
autoinstall:
  version: 1
  identity:
    hostname: srv01
    username: admin
    password: '$6$rounds=4096$saltsalt$placeholderHash'   # mkpasswd -m sha-512
  ssh:
    install-server: true
    allow-pw: false
    authorized-keys:
      - ssh-ed25519 AAAA... admin@workstation
  storage:
    layout:
      name: lvm
  packages:
    - openssh-server
    - chrony
    - prometheus-node-exporter
  late-commands:
    - curtin in-target -- systemctl enable chrony
```

```
EOF
touch meta-data
genisoimage -output seed.iso -volid cidata -joliet -rock user-data meta-data
ls -lh seed.iso
```

On a real install, attach `seed.iso` as a virtual CD via IP KVM and boot the Ubuntu Server ISO with `autoinstall ds=nocloud-net` on the kernel command line. The install runs end-to-end with **no operator interaction**.

Step 4 — Imaging and cloning — capture a golden image

Note: This will take more than 10-15 mins based on the size of the image

Take a snapshot of the running root filesystem as a tarball you can deploy elsewhere:

```
tar --one-file-system --exclude=/proc --exclude=/sys --exclude=/dev --exclude=/mnt \
    -czf /srv/golden.tgz / 2>/dev/null
ls -lh /srv/golden.tgz
```

This is the basic **physical image** pattern. For **VM cloning** the equivalent is copying the `.qcow2`/`.vmdk`. For both, run a "sysprep" step (`cloud-init clean`, regenerate SSH host keys, clear `/etc/machine-id`) before re-deploying.

Step 5 — Template deployment with debootstrap (bare-metal style)

`debootstrap` builds a Linux root filesystem from scratch — the same mechanism behind container images and template deployment:

```
mkdir -p /srv/template
debootstrap --variant=minbase jammy /srv/template http://archive.ubuntu.com/ubuntu/ 2>&1 | tail
du -sh /srv/template
```

`/srv/template` is now a deployable filesystem you could tar, ship, and extract onto any new host.

Step 6 — P2V (Physical to Virtual) conversion concept

The standard P2V workflow:

1. **Inventory** the source server (CPU, RAM, disks, NICs, MAC for licence binding).
2. **Quiesce** services or take a consistent backup.
3. **Image** each disk with `dd` or `qemu-img convert` directly to `.qcow2`/`.vmdk`.
4. **Inject drivers** for the hypervisor's virtual NIC and disk controller.
5. **Power on** in the hypervisor and verify network, licensing, and apps.

```
qemu-img create -f qcow2 /srv/converted.qcow2 5G   # destination image shape
qemu-img info /srv/converted.qcow2
```

Step 7 — GUI vs. Core vs. Bare metal vs. Virtualized vs. Remote

Install type	When to use
GUI	Admin workstation, training labs
Core	Production server — smaller attack surface, fewer patches
Bare metal	Single-tenant, max performance, hardware-dependent workloads
Virtualized	Most production today — consolidation, snapshots, live migrate
Remote	Branch sites, lights-out data centres — via PXE / IP KVM / cloud-init

Free online tools

- **Ubuntu autoinstall reference** — https://ubuntu.com/server/docs/install/autoinstall
- **Red Hat Kickstart** — https://access.redhat.com/documentation/
- **cloud-init docs** — https://cloudinit.readthedocs.io/
- **OS HCL portals** — Red Hat, SUSE, Ubuntu Certified, VMware Compatibility Guide

What you learned

- Pre-install validation: minimum spec, architecture, HCL check.
- Unattended install with a cloud-init `seed.iso` (modern preseed/kickstart equivalent).
- Imaging, cloning, sysprep, and the P2V conversion workflow.
- When to choose GUI vs. Core vs. Bare metal vs. Virtualized vs. Remote install types.